

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Nania Aurelia Dewi Cahyani - 5024241059

2026

1 Langkah-Langkah Percobaan

1.1 Persiapan Awal

1. Menyiapkan *board* STM32 Blackpill beserta ST-Link V2 dan *board* ESP32 DevKit tanpa menyambungkannya ke komputer terlebih dahulu.

2. Membuat satu *project* baru pada PlatformIO di Visual Studio Code (misalnya dengan nama `modul1-praktikum`) yang digunakan untuk seluruh rangkaian Percobaan 1 hingga 5 dengan mengganti isi file `platformio.ini` serta kode program utama sesuai kebutuhan tiap percobaan.

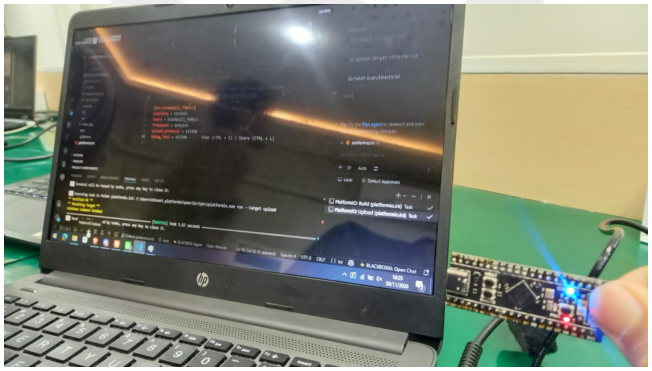
1.2 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Mengubah konfigurasi file `platformio.ini` untuk target *board* `blackpill_f401cc` dengan *framework* `arduino` dan protokol unggah `stlink`, lalu menunggu hingga proses *refresh* konfigurasi PlatformIO selesai.

2. Menghubungkan pin ST-Link V2 ke *board* STM32 Blackpill sesuai pasangan pinnya (`SWDIO` ke `SWDIO`, `SWCLK` ke `SWCLK`, `GND` ke `GND`, dan `3.3V` ke `3V3`), kemudian menghubungkan ST-Link ke port USB komputer.

3. Menuliskan kode program *Blink* untuk pin LED *onboard* `PC13` pada file `src/main.cpp`.

4. Melakukan proses *Build* dan *Upload* program ke mikrokontroler, lalu mengamati kedipan LED *onboard* `PC13` dengan interval 500 ms.



Gambar 1: Percobaan 1

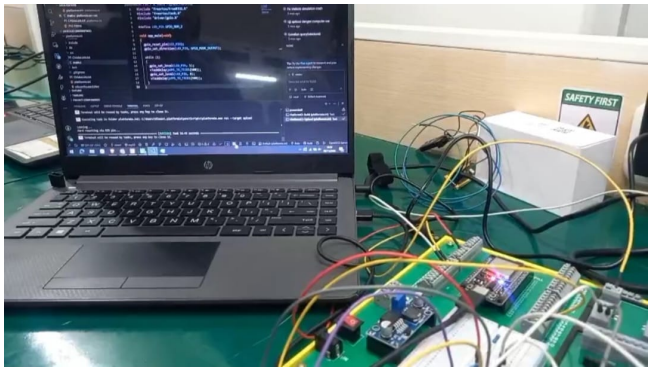
1.3 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Mengubah isi file `platformio.ini` untuk target *board* `esp32dev` dengan *framework* `espidf`, lalu menunggu hingga proses pembaruan indeks selesai.

2. Menghubungkan *board* ESP32 DevKit ke komputer menggunakan kabel USB dan memastikan port serial terbaca oleh sistem.

3. Menghapus file `src/main.cpp`, kemudian membuat file baru `src/main.c` dan menuliskan kode program *Blink* berbasis FreeRTOS untuk pin `GPIO_NUM_2`.

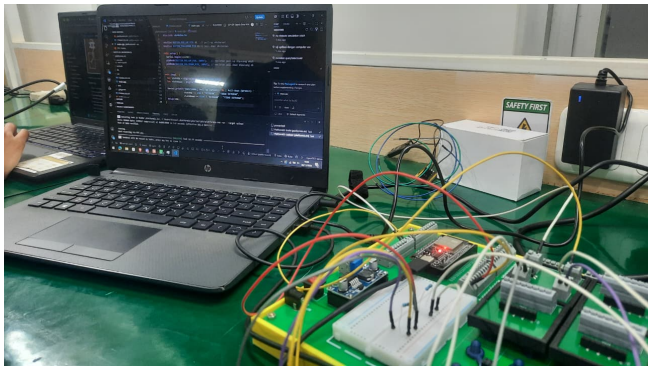
4. Melakukan proses *Build* dan *Upload*, lalu mengamati nyala LED *onboard* serta menganalisis perbedaan struktur program ESP-IDF dibandingkan dengan *framework* Arduino.



Gambar 2: Percobaan 2

1.4 Percobaan 3: Pull-up dan Pull-down: Eksternal vs Internal

1. Mengembalikan konfigurasi `platformio.ini` ke *framework* `arduino`, menghapus `src/main.c`, serta membuat kembali file `src/main.cpp`.
2. Merangkai dua buah *pushbutton* pada *breadboard* menuju GPIO 32 (konfigurasi *pull-up* eksternal ke 3.3V melalui resistor $10\,\mathrm{k}\Omega$) dan GPIO 33 (konfigurasi *pull-down* eksternal ke GND melalui resistor $10\,\mathrm{k}\Omega$).
3. Menuliskan kode program pengujian konfigurasi eksternal pada `src/main.cpp`, melakukan *Build* dan *Upload*, lalu menekan kedua tombol sambil mencatat hasil pembacaan pada Serial Monitor.
4. Melepas kedua resistor eksternal $10\,\mathrm{k}\Omega$ dari rangkaian dengan tetap mempertahankan posisi kedua tombol pada GPIO 32 dan GPIO 33.
5. Mengganti kode pada `src/main.cpp` untuk mengaktifkan resistor internal melalui mode `INPUT_PULLUP` dan `INPUT_PULLDOWN`, melakukan *Build* dan *Upload*, lalu membandingkan hasil keluaran Serial Monitor dengan pengujian eksternal sebelumnya.



Gambar 3: Percobaan 3

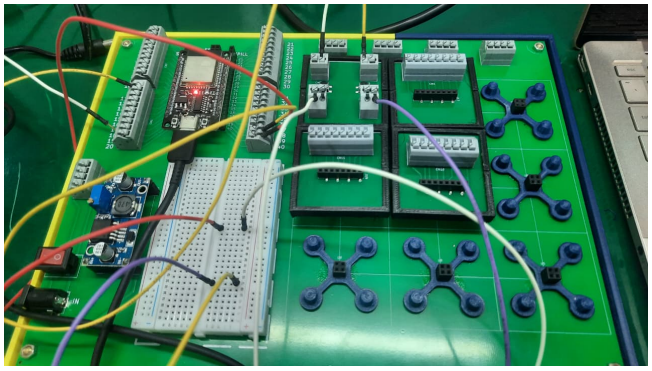
1.5 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Merangkai satu buah *pushbutton* pada *breadboard* dengan menghubungkan satu kakinya ke GPIO 25 dan kaki lainnya ke GND.

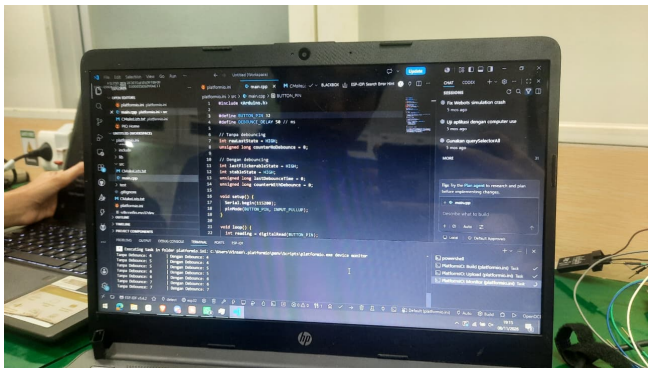
2. Menuliskan kode program perbandingan penghitung tanpa *debouncing* (`counterNoDebounce`) dan dengan *debouncing* (`counterWithDebounce`) menggunakan mode `INPUT_PULLUP` dan nilai awal `DEBOUNCE_DELAY` sebesar 50 ms.

3. Melakukan *Build* dan *Upload*, kemudian menekan tombol dengan variasi kecepatan (lambat dan cepat) sambil mengamati perbandingan kenaikan kedua nilai penghitung pada Serial Monitor.

4. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms dan 200 ms secara bergantian, melakukan *Upload* ulang pada setiap perubahan, serta membandingkan akurasi penyaringan getaran dan responsivitas tombol dari ketiga variasi waktu tersebut.



Gambar 4: Percobaan 4



Gambar 5: Percobaan 4

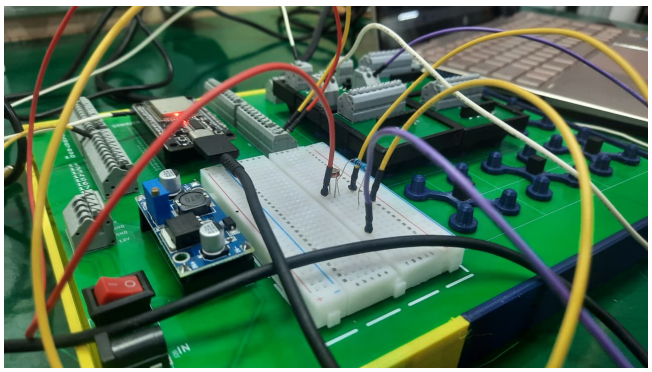
1.6 Percobaan 5: Akses GPIO Analog: Pembacaan ADC dengan LDR

1. Merangkai sensor LDR secara seri dengan resistor $10\,\text{k}\Omega$ sebagai pembagi tegangan (kaki LDR ke 3.3V dan kaki resistor ke GND), lalu menghubungkan titik tengah keduanya ke pin GPIO 34.

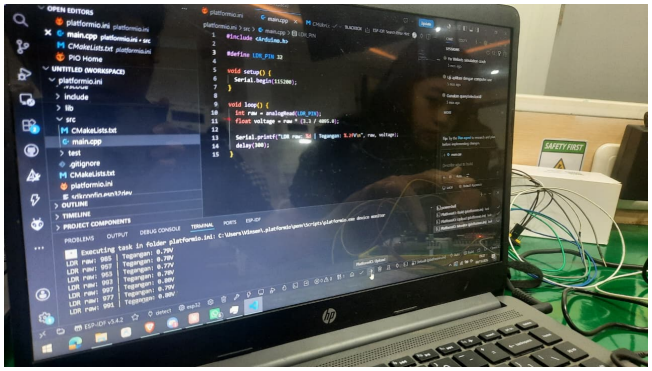
2. Menuliskan kode program pada `src/main.cpp` untuk membaca nilai mentah ADC 12-bit menggunakan fungsi `analogRead(34)` serta mengonversinya menjadi nilai tegangan (0–$3.3\,\text{V}$).

3. Melakukan proses *Build* dan *Upload*, lalu membuka Serial Monitor.

4. Memberikan variasi intensitas cahaya dengan menutup permukaan LDR menggunakan tangan dan menyinarinya menggunakan lampu, kemudian mengamati perubahan nilai mentah ADC (0–4095) serta tegangan yang terbaca.



Gambar 6: Percobaan 5



Gambar 7: Percobaan 5

2 Analisis Hasil Percobaan

Pada Percobaan 1 dan Percobaan 2, proses flashing program Blink berhasil dijalankan pada dua arsitektur mikrokontroler yang berbeda menggunakan satu lingkungan kerja terpadu di PlatformIO. Pada STM32 Blackpill, penggunaan framework Arduino (STM32duino) mempermudah inisialisasi pin fisik secara langsung (PC13) tanpa perlu mengonfigurasi register HAL secara manual, di mana LED onboard menyala saat diberi logika LOW dan mati saat logika HIGH sesuai dengan karakteristik rangkaian aktif-LOW pada teori. Sementara itu, pada Percobaan 2 dengan board ESP32 menggunakan framework native ESP-IDF, program berjalan di atas sistem operasi waktu nyata (FreeRTOS) dengan titik masuk app_main() serta memanfaatkan fungsi vTaskDelay() yang tidak memblokir task lainnya, berbeda dengan fungsi delay() pada Arduino. Keberhasilan kedua percobaan ini dipengaruhi oleh ketepatan konfigurasi pada file platformio.ini, kelengkapan instalasi driver USB (ST-Link untuk STM32 dan CP2102/CH340 untuk ESP32), serta ketepatan pemasangan kabel jalur SWD (SWDIO, SWCLK, GND, dan 3.3V) pada Blackpill agar proses unggah program tidak gagal.

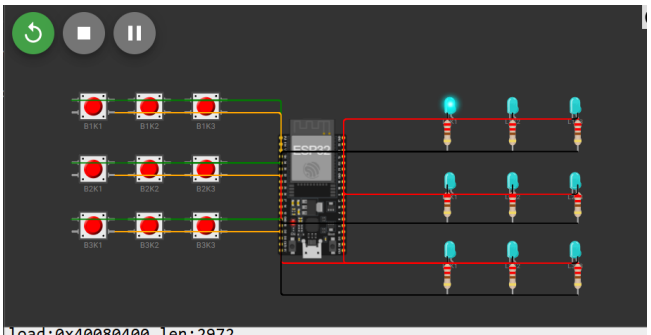
Pada Percobaan 3, pengujian konfigurasi resistor pull-up dan pull-down baik secara eksternal maupun internal pada ESP32 menghasilkan perilaku logika yang sesuai dengan dasar teori. Saat menggunakan konfigurasi pull-up pada GPIO 32, kondisi awal tombol yang tidak ditekan terbaca HIGH karena pin terhubung ke tegangan 3.3V melalui resistor, dan berubah menjadi LOW ketika tombol ditekan karena jalur terhubung langsung ke GND; sebaliknya, konfigurasi pull-down pada GPIO 33 menghasilkan pembacaan LOW saat tidak ditekan dan HIGH saat ditekan. Perbandingan antara rangkaian eksternal (menggunakan resistor fisik 10k) dan internal (mengaktifkan INPUT_PULLUP serta INPUT_PULLDOWN lewat program) menunjukkan keluaran yang identik pada Serial Monitor,

sehingga terbukti bahwa resistor internal dapat menggantikan komponen eksternal untuk mencegah kondisi floating pada pin input. Apabila terjadi ketidaksesuaian pembacaan selama praktikum, hal tersebut umumnya disebabkan oleh kesalahan dalam menentukan pasangan kaki pada pushbutton tactile empat kaki atau resistor eksternal yang belum dilepas sepenuhnya saat beralih ke pengujian mode internal.

Pada Percobaan 4, pengamatan perbedaan antara penghitung tanpa debouncing (counterNoDebounce) dan dengan debouncing (counterWithDebounce) membuktikan adanya fenomena bouncing akibat getaran mekanis pada kontak logam di dalam pushbutton saat ditekan maupun dilepas. Tanpa penyaring waktu, satu kali penekanan tombol sering kali memicu lonjakan angka lebih dari satu pada counterNoDebounce karena mikrokontroler membaca transisi HIGH ke LOW yang berulang dalam hitungan milidetik. Sebaliknya, algoritma debouncing berbasis fungsi millis() berhasil menyaring sinyal palsu tersebut dengan memastikan perubahan status hanya dicatat setelah sinyal stabil melewati batas DEBOUNCE_DELAY. Pengujian variasi nilai jeda (10 ms, 50 ms, dan 200 ms) menunjukkan adanya trade-off antara akurasi dan responsivitas: nilai jeda yang terlalu kecil (10 ms) masih berisiko meloloskan getaran kontak, sedangkan nilai yang terlalu besar (200 ms) membuat sistem terasa lambat atau mengabaikan penekanan tombol yang dilakukan secara cepat berturut-turut, sehingga nilai di kisaran 50 ms menjadi titik keseimbangan yang paling optimal.

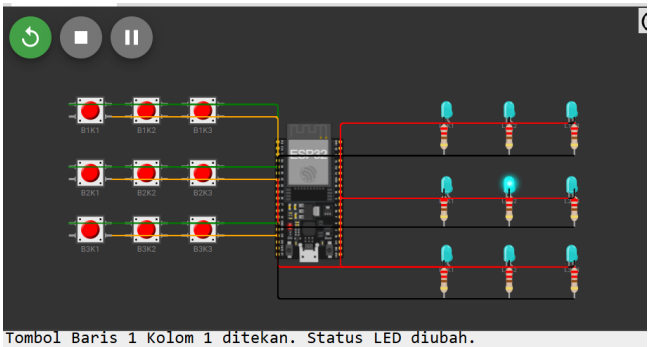
Pada Percobaan 5, pembacaan sensor cahaya LDR melalui pin GPIO 34 menunjukkan cara kerja ADC (Analog-to-Digital Converter) sebagai mode akses GPIO ketiga yang mampu menangkap perubahan tegangan secara kontinu, berbeda dengan input digital yang hanya mengenal dua kondisi biner (HIGH atau LOW). Karena dirangkai sebagai pembagi tegangan bersama resistor tetap 10k, perubahan intensitas cahaya yang mengenai permukaan LDR mengubah nilai resistansinya, sehingga tegangan pada titik tengah yang masuk ke pin ADC ikut bervariasi dan dikonversi oleh ADC 12-bit ESP32 menjadi nilai digital rentang 0 hingga 4095 (setara 0V hingga 3.3V). Ketika LDR ditutup dengan tangan, resistansi LDR meningkat sehingga pembagian tegangan berubah dan nilai pembacaan di Serial Monitor bergeser secara drastis dibandingkan saat LDR disinari langsung oleh lampu. Keakuratan hasil pembacaan analog ini sangat dipengaruhi oleh kestabilan sumber tegangan 3.3V, toleransi nilai resistor pembagi tegangan, serta pemilihan pin GPIO 34 yang merupakan bagian dari kanal ADC1 sehingga pembacaannya tidak terganggu oleh aktivitas modul WiFi internal ESP32.

3 Hasil Tugas Modul



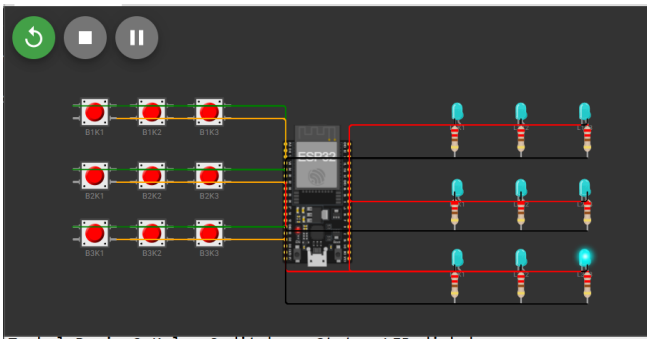
```
load:0x40080400,len:2972
entry 0x400805dc
Tombol Baris 1 Kolom 1 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
```

Gambar 8: Simulasi



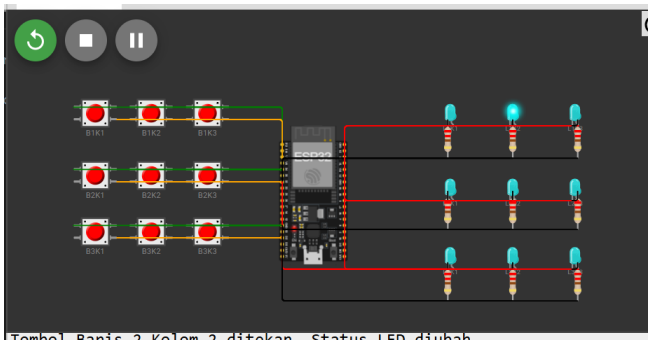
```
Tombol Baris 1 Kolom 1 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 1 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
```

Gambar 9: Simulasi



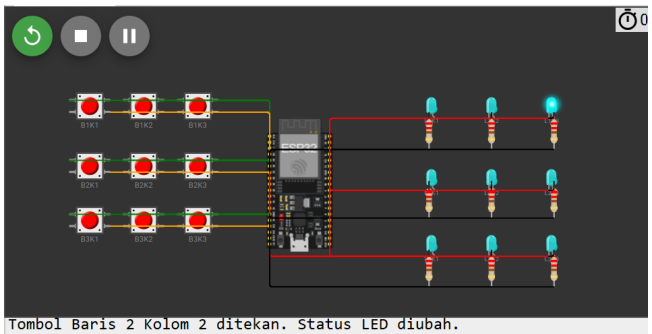
```
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 1 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
```

Gambar 10: Simulasi



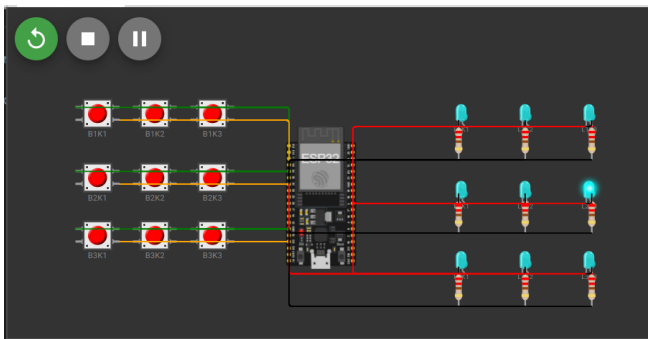
```
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 1 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 2 ditekan. Status LED diubah.
```

Gambar 11: Simulasi



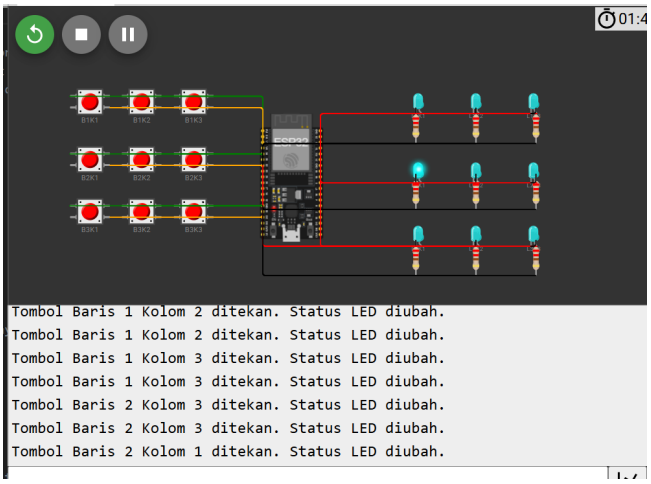
```
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 3 ditekan. Status LED diubah.
```

Gambar 12: Simulasi

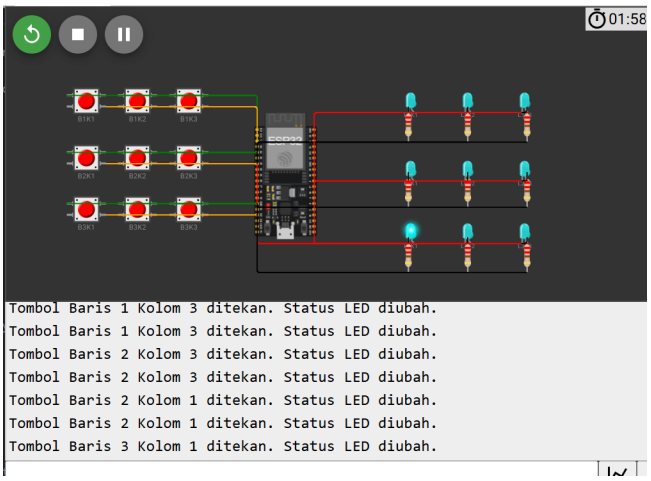


```
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 3 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 2 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 1 Kolom 3 ditekan. Status LED diubah.
Tombol Baris 2 Kolom 3 ditekan. Status LED diubah.
```

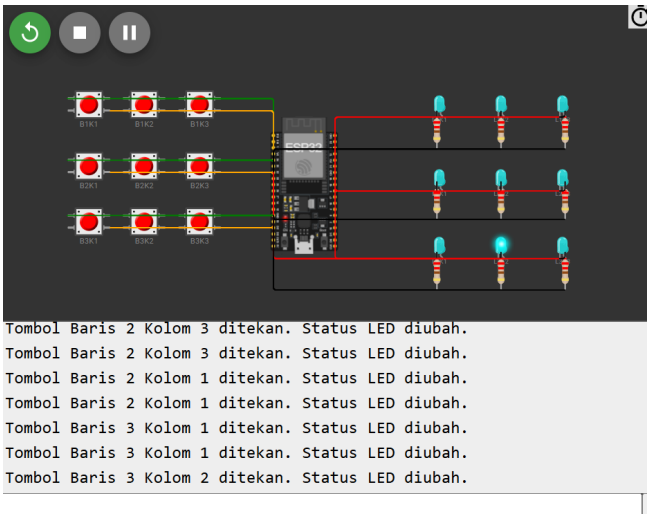
Gambar 13: Simulasi



Gambar 14: Simulasi



Gambar 15: Simulasi



Gambar 16: Simulasi

3.1 Jawaban Pertanyaan Tugas Modul

1. Meskipun pada satu waktu umumnya hanya satu tombol yang ditekan, fenomena bouncing terjadi secara fisiko-mekanis pada setiap tombol tactile akibat getaran elastis kontak logam saat

bersentuhan atau terlepas. Tanpa adanya algoritma debouncing, pemindaian matriks (matrix scanning) yang berkecepatan tinggi dapat menangkap puluhan transisi sinyal HIGH-LOW palsu dalam kurun waktu beberapa milidetik untuk satu kali penekanan. Hal ini akan menyebabkan status LED berkedip tak beraturan atau memicu perubahan state berulang kali secara tidak sengaja.

2. Pada rancangan di atas, pin kolom dikonfigurasi menggunakan INPUT_PULLUP internal (yang setara dengan menghubungkan pin ke VCC 3.3V melalui resistor internal). Saat tombol tidak ditekan, pin kolom berada pada kondisi HIGH karena terisolasi dari jalur ground baris dan ditarik ke tegangan positif. Kondisi ini tidak boleh dibiarkan floating (mengambang tanpa referensi tegangan yang pasti) karena jalur pin input yang terbuka akan bertindak sebagai antena yang menangkap noise elektromagnetik di sekitarnya, mengakibatkan mikrokontroler membaca status tombol berubah-ubah secara acak (false triggering).

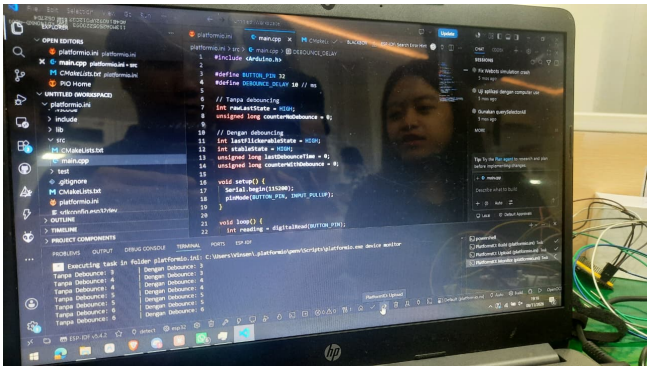
3. Digital Input (seperti pada pembacaan tombol) hanya mengenali dua tingkat diskrit, yaitu HIGH (1) atau LOW (0), sehingga tidak mampu mengukur gradasi atau variasi tingkat kondisi di antaranya. Analog Input melalui ADC (Analog-to-Digital Converter) mampu membaca tegangan kontinu (seperti perubahan tegangan pada pembagi resistor LDR akibat intensitas cahaya) dan mengubahnya menjadi rentang nilai digital yang luas—pada ESP32 menggunakan resolusi 12-bit bernilai 0 hingga 4095. ADC dapat merepresentasikan jauh lebih dari dua kondisi karena ia mengkuantitasi nilai tegangan analog proporsional terhadap referensi sistem (0V–3.3V), sehingga setiap perbedaan intensitas cahaya sekecil apa pun menghasilkan angka diskrit yang unik.

4 Kesimpulan

Berdasarkan seluruh rangkaian praktikum yang telah dilaksanakan, dapat disimpulkan bahwa pemahaman mengenai arsitektur perangkat keras serta pemilihan framework pengembangan yang tepat, seperti penggunaan framework Arduino pada STM32 Blackpill dan ESP-IDF maupun Arduino pada ESP32, merupakan fondasi utama dalam keberhasilan flashing dan eksekusi program mikrokontroler. Selain itu, pemanfaatan konfigurasi resistor pull-up dan pull-down (baik eksternal maupun internal) terbukti efektif dalam mencegah kondisi floating pada pin input digital. Implementasi algoritma debouncing berbasis waktu juga sangat krusial untuk menyaring getaran mekanis kontak tombol sehingga pembacaan input menjadi akurat, sementara penggunaan ADC melengkapi pemahaman akses GPIO sebagai input analog yang mampu membaca variasi tegangan secara kontinu, seperti pada pemanfaatan sensor LDR.

5 Lampiran

5.1 Dokumentasi saat praktikum



Gambar 17: Lampiran